FYEO

Security Assessment of Iagon.org Staking Contract

Iagon.org

November 2023
Version 1.0

Presented by:
FYEO Inc.
PO Box 147044
Lakewood CO 80214
United States

Security Level
Public

Table of Contents

Executive Summary 2
- Overview 2
- Key Findings 2
- Scope and Rules of Engagement 2

Technical Analyses and Findings 4
- Findings 5
- Technical Analysis 5
- Conclusion 5
- Technical Findings 6
  - General Observations 6
  - staking.ak, reward.ak: testing is lacking 7
  - staking.ak: more state transition validation for redeemers Withdraw and Resize 8

Our Process 10
- Methodology 10
  - Kickoff 10
  - Ramp-up 10
  - Review 11
  - Code Safety 11
  - Technical Specification Matching 11
  - Reporting 12
  - Verify 12
- Additional Note 12
- The Classification of vulnerabilities 13

EXECUTIVE SUMMARY

Overview

Iagon.org engaged FYEO Inc. to perform a Security Assessment of Iagon.org Staking Contract.

The assessment was conducted remotely by the FYEO Security Team. Testing took place on October 31 - November 18, 2023, and focused on the following objectives:

- To provide the customer with an assessment of their overall security posture and any risks that were discovered within the environment during the engagement.
- To provide a professional opinion on the maturity, adequacy, and efficiency of the security measures that are in place.
- To identify potential issues and include improvement recommendations based on the results of our tests.

This report summarizes the engagement, tests performed, and findings. It also contains detailed descriptions of the discovered vulnerabilities, steps the FYEO Security Team took to identify and validate each issue, as well as any applicable recommendations for remediation.

Key Findings

The following issues have been identified during the testing period. These should be prioritized for remediation to reduce the risk they pose:

- FYEO-IAGO-ID-01 – staking.ak, reward.ak: testing is lacking; **STATUS - REMEDIATED**
- FYEO-IAGO-ID-02 – staking.ak: more state transition validation for redeemers Withdraw and Resize; **STATUS - REMEDIATED**

Based on our review process, we conclude that the reviewed code implements the documented functionality.

Scope and Rules of Engagement

The FYEO Review Team performed a Security Assessment of Iagon.org Staking Contract. The following table documents the targets in scope for the engagement. No additional systems or resources were in scope for this assessment.

The source code was supplied through a private repository at https://github.com/Iagonorg/staking-contract/ with the commit hash main.

Files included in the code review
staking-contract/ ├── lib/ │ └── staking/ │ ├── helpers.ak │ └── types.ak └── validators/ ├── reward.ak └── staking.ak

Table 1: Scope

TECHNICAL ANALYSES AND FINDINGS

During the Security Assessment of Iagon.org Staking Contract, we discovered:

- 2 findings with LOW severity rating.

The following chart displays the findings by severity.

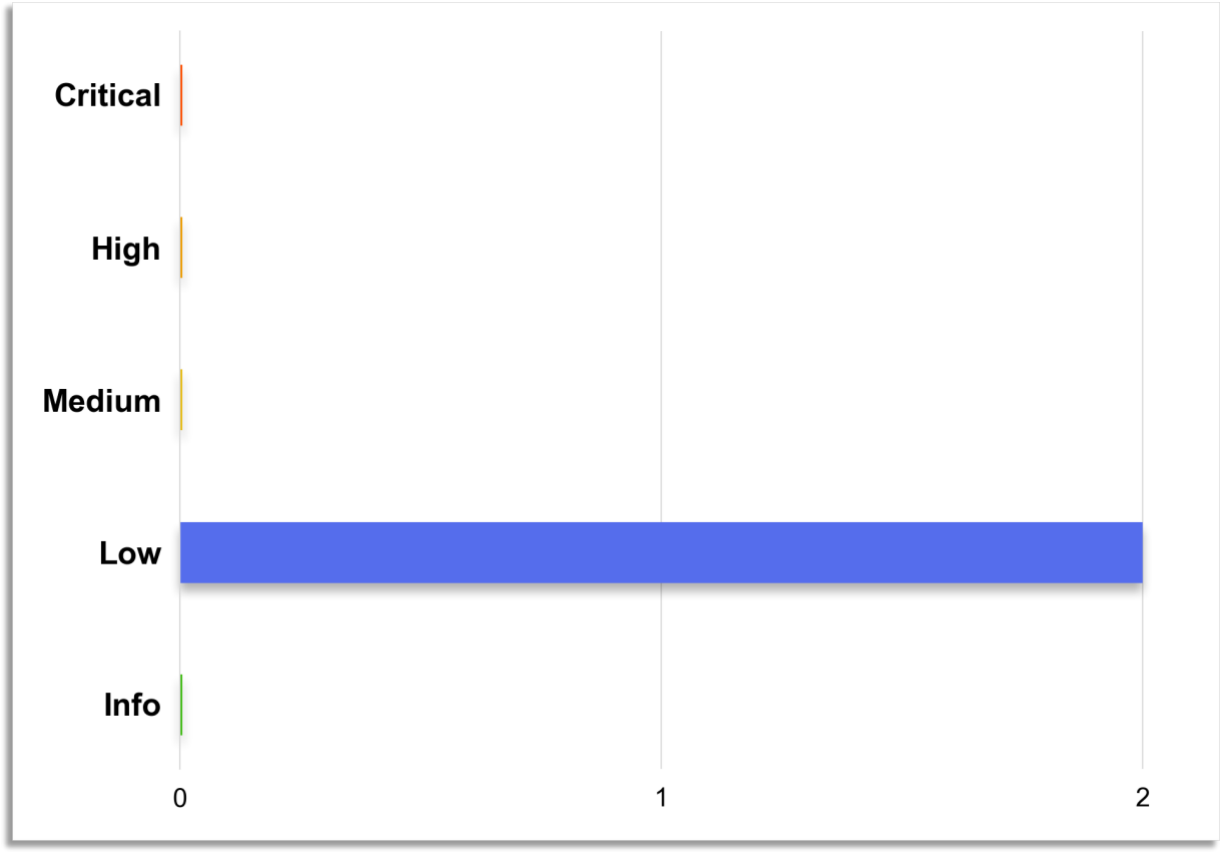


Figure 1: Findings by Severity

FINDINGS

The *Findings* section provides detailed information on each of the findings, including methods of discovery, explanation of severity determination, recommendations, and applicable references.

The following table provides an overview of the findings.

Finding #	Severity	Description
FYEO-IAGO-ID-01	**Low**	staking.ak, reward.ak: testing is lacking
FYEO-IAGO-ID-02	**Low**	staking.ak: more state transition validation for redeemers Withdraw and Resize

Table 2: Findings Overview

TECHNICAL ANALYSIS

The source code has been manually validated to the extent that the state of the repository allowed. The validation includes confirming that the code correctly implements the intended functionality.

CONCLUSION

Based on our review process, we conclude that the code implements the documented functionality to the extent of the reviewed code.

Technical Findings

General Observations

The code is well-written and well-documented. The presence of the architecture explainer has been a much welcome gesture, and greatly assisted in the analysis of possible threat models and understanding the intent of the validators.

FYEO did not identify any high or critical findings that raises serious concerns or could be considered a vulnerability.

In general, since all transactions must be signed off by the backend, there is little wiggle room for a potential attacker to break the security. However, it is still a good idea to thoroughly validate the state transitions of the inputs and outputs regardless of whether the backend signed off on them or not, and this is arguably the main conclusion that can be made from this analysis.

After initial comments from the team has developed extensive unit tests that covers the majority of cases including edge cases and improved the state validation.

The team worked diligently to implement fixes for even the informational findings in the audit and the fixes have been reviewed and are of good quality.

STAKING.AK, REWARD.AK: TESTING IS LACKING

Finding ID: FYEO-IAGO-ID-01
Severity: **Low**
Status: **Remediated**

Description

The primary purpose of Aiken validators is to validate transactions by examining inputs and outputs. It is therefore highly recommended to thoroughly test these state transitions, especially with negative test cases to ensure that transactions that should be rejected are indeed rejected.

Severity and Impact Summary

The presence of a comprehensive test suite always allows for a higher degree of confidence that the code functions correctly, as well as guards against potential bugs in the future.

Recommendation

Create a test suite around different input/output combinations and ensure that they pass/fail as expected.

STAKING.AK: MORE STATE TRANSITION VALIDATION FOR REDEEMERS WITHDRAW AND RESIZE

Finding ID: FYEO-IAGO-ID-02
Severity: **Low**
Status: **Remediated**

Description

Some state transition checks for output datums appear lacking or unclear.

Proof of Issue

File name: staking.ak
Line number: 113-156

```
    when redeemer is {
      Retire(_, _) -> {
        expect [own_output] = own_outputs
        expect Some(stk_utxo_lock_until) = certificate.stk_utxo_lock_until
        let op_qty =
          value.quantity_of(own_output.value, tkn_policy_id, tkn_asset_name)

        helpers.is_tx_after(ctx.transaction.validity_range, datum.locked_until) &&
valid_output_datum(
          own_output,
          msg,
        ) && datum.state == Active && output_stake_datum.state == Retiring && op_qty
== input_tkn_qty && output_stake_datum.locked_until == stk_utxo_lock_until &&
datum.token == output_stake_datum.token
      }

      Withdraw(_, _) -> {
        let cert_tkn_qty =
          value.quantity_of(certificate.value, tkn_policy_id, tkn_asset_name)

        // TODO: Need to decide what to do with penalty
        // For now we are sending penalty to address defined in params
        helpers.is_tx_after(ctx.transaction.validity_range, datum.locked_until) &&
datum.state == Retiring && check_penalty_amount(
          ctx.transaction.outputs,
          input_tkn_qty,
          cert_tkn_qty,
          tkn_policy_id,
          tkn_asset_name,
          penalty_addr,
        )
      }

      Resize(_, _) -> {
        expect [own_output] = own_outputs
        let cert_tkn_qty =
          value.quantity_of(certificate.value, tkn_policy_id, tkn_asset_name)
```

```
        let op_qty =
          value.quantity_of(own_output.value, tkn_policy_id, tkn_asset_name)

        op_qty == cert_tkn_qty && op_qty > input_tkn_qty && output_stake_datum.token
== datum.token && valid_output_datum(
          own_output,
          msg,
        ) && datum.state == Active && output_stake_datum.state == Active && Some(
          output_stake_datum.locked_until,
        ) == certificate.stk_utxo_lock_until
      }
    }
```

Severity and Impact Summary

Generally, since output datums have to be signed by the backend as a precondition to any operations, the attack surface is very low. This means that any incorrect state transitions can be only introduced with the explicit approval of the backend, and so an error in the backend logic has to be discovered. However, it is still a possibility.

Recommendation

It is best to be as explicit and transparent with the state transition between input and output datums as possible. This helps promote clarity if the validator logic is used as a reference, as well as guard against any logic errors elsewhere in the system.

Concretely, it would be a good idea to create validation functions in the form of: `validate_<operation>_datum_transition(input, output, ...)`, and ensure that every property of both input and output are validated, whether they are supposed to change or not (e.g. datum.state going from Retiring to Retiring, or Active to Retiring, whether datum.locked_until changed, etc.). In general, it is good to assume that an attacker has somehow managed to get the backend to sign off on a transaction that could be considered invalid - then those extra transition validations will thwart the attacker regardless of the compromised status of the backend.

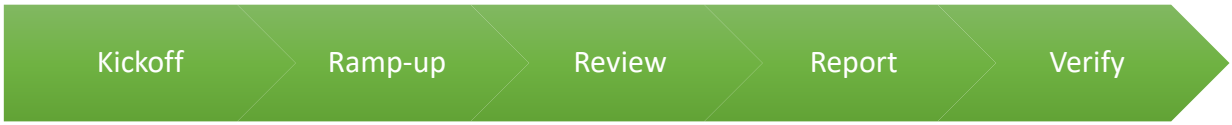
Another recommendation is to check *all* transaction outputs, especially for `Withdraw` to ensure that unexpected outputs are not being created - i.e. only the `StakeDatum` exists alongside withdraw destinations.

OUR PROCESS

METHODOLOGY

FYEO Inc. uses the following high-level methodology when approaching engagements. They are broken up into the following phases.

Figure 2: Methodology Flow



KICKOFF

The project is kicked off as the sales process has concluded. We typically set up a kickoff meeting where project stakeholders are gathered to discuss the project as well as the responsibilities of participants. During this meeting we verify the scope of the engagement and discuss the project activities. It's an opportunity for both sides to ask questions and get to know each other. By the end of the kickoff there is an understanding of the following:

- Designated points of contact
- Communication methods and frequency
- Shared documentation
- Code and/or any other artifacts necessary for project success
- Follow-up meeting schedule, such as a technical walkthrough
- Understanding of timeline and duration

RAMP-UP

Ramp-up consists of the activities necessary to gain proficiency on the project. This can include the steps needed for familiarity with the codebase or technological innovation utilized. This may include, but is not limited to:

- Reviewing previous work in the area including academic papers
- Reviewing programming language constructs for specific languages
- Researching common flaws and recent technological advancements

Review

The review phase is where most of the work on the engagement is completed. This is the phase where we analyze the project for flaws and issues that impact the security posture. Depending on the project this may include an analysis of the architecture, a review of the code, and a specification matching to match the architecture to the implemented code.

In this code audit, we performed the following tasks:

1. Security analysis and architecture review of the original protocol
2. Review of the code written for the project
3. Compliance of the code with the provided technical documentation

The review for this project was performed using manual methods and utilizing the experience of the reviewer. No dynamic testing was performed, only the use of custom-built scripts and tools were used to assist the reviewer during the testing. We discuss our methodology in more detail in the following sections.

Code Safety

We analyzed the provided code, checking for issues related to the following categories:

- General code safety and susceptibility to known issues
- Poor coding practices and unsafe behavior
- Leakage of secrets or other sensitive data through memory mismanagement
- Susceptibility to misuse and system errors
- Error management and logging

This list is general and not comprehensive, meant only to give an understanding of the issues we are looking for.

Technical Specification Matching

We analyzed the provided documentation and checked that the code matches the specification. We checked for things such as:

- Proper implementation of the documented protocol phases
- Proper error handling
- Adherence to the protocol logical description

Reporting

FYEO Inc. delivers a draft report that contains an executive summary, technical details, and observations about the project.

The executive summary contains an overview of the engagement including the number of findings as well as a statement about our general risk assessment of the project. We may conclude that the overall risk is low but depending on what was assessed we may conclude that more scrutiny of the project is needed.

We report security issues identified, as well as informational findings for improvement, categorized by the following labels:

- Critical
- High
- Medium
- Low
- Informational

The technical details are aimed more at developers, describing the issues, the severity ranking and recommendations for mitigation.

As we perform the audit, we may identify issues that aren't security related, but are general best practices and steps that can be taken to lower the attack surface of the project. We will call those out as we encounter them and as time permits.

As an optional step, we can agree on the creation of a public report that can be shared and distributed with a larger audience.

Verify

After the preliminary findings have been delivered, this could be in the form of the approved communication channel or delivery of the draft report, we will verify any fixes within a window of time specified in the project. After the fixes have been verified, we will change the status of the finding in the report from open to remediated.

The output of this phase will be a final report with any mitigated findings noted.

Additional Note

It is important to note that, although we did our best in our analysis, no code audit or assessment is a guarantee of the absence of flaws. Our effort was constrained by resource and time limits along with the scope of the agreement.

While assessing the severity of the findings, we considered the impact, ease of exploitability, and the probability of attack. This is a solid baseline for severity determination.

THE CLASSIFICATION OF VULNERABILITIES

Security vulnerabilities and areas for improvement are weighted into one of several categories using, but is not limited to, the criteria listed below:

Critical – vulnerability will lead to a loss of protected assets

- This is a vulnerability that would lead to immediate loss of protected assets
- The complexity to exploit is low
- The probability of exploit is high

High - vulnerability has potential to lead to a loss of protected assets

- All discrepancies found where there is a security claim made in the documentation that cannot be found in the code
- All mismatches from the stated and actual functionality
- Unprotected key material
- Weak encryption of keys
- Badly generated key materials
- Txn signatures not verified
- Spending of funds through logic errors
- Calculation errors overflows and underflows

Medium - vulnerability hampers the uptime of the system or can lead to other problems

- Insecure calls to third party libraries
- Use of untested or nonstandard or non-peer-reviewed crypto functions
- Program crashes, leaves core dumps or writes sensitive data to log files

Low – vulnerability has a security impact but does not directly affect the protected assets

- Overly complex functions
- Unchecked return values from 3rd party libraries that could alter the execution flow

Informational

- General recommendations